

CU-423 RoboLink Programming - Introduction

Dr. Mahmoud **Sayed** & Mahmoud **Yassin**

05-May-2026

What are we doing today?

- Get to know what a computer *really* is.
- Binary system?
- C/C++ basic concepts
 - Variables
 - Loops
 - Functions

A computer: Hardware & Software

- Hardware: Physical parts of the computer
 - Keyboard, monitor, processor,...
- Software: Programs and data: Series of Instructions.

Hardware vs Software

Hardware

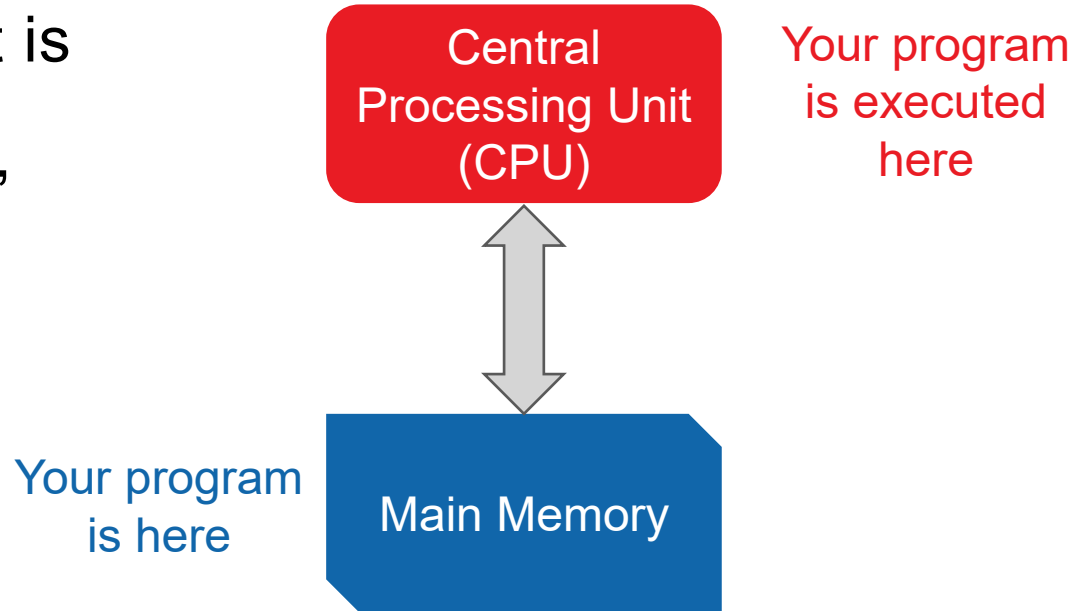


Software

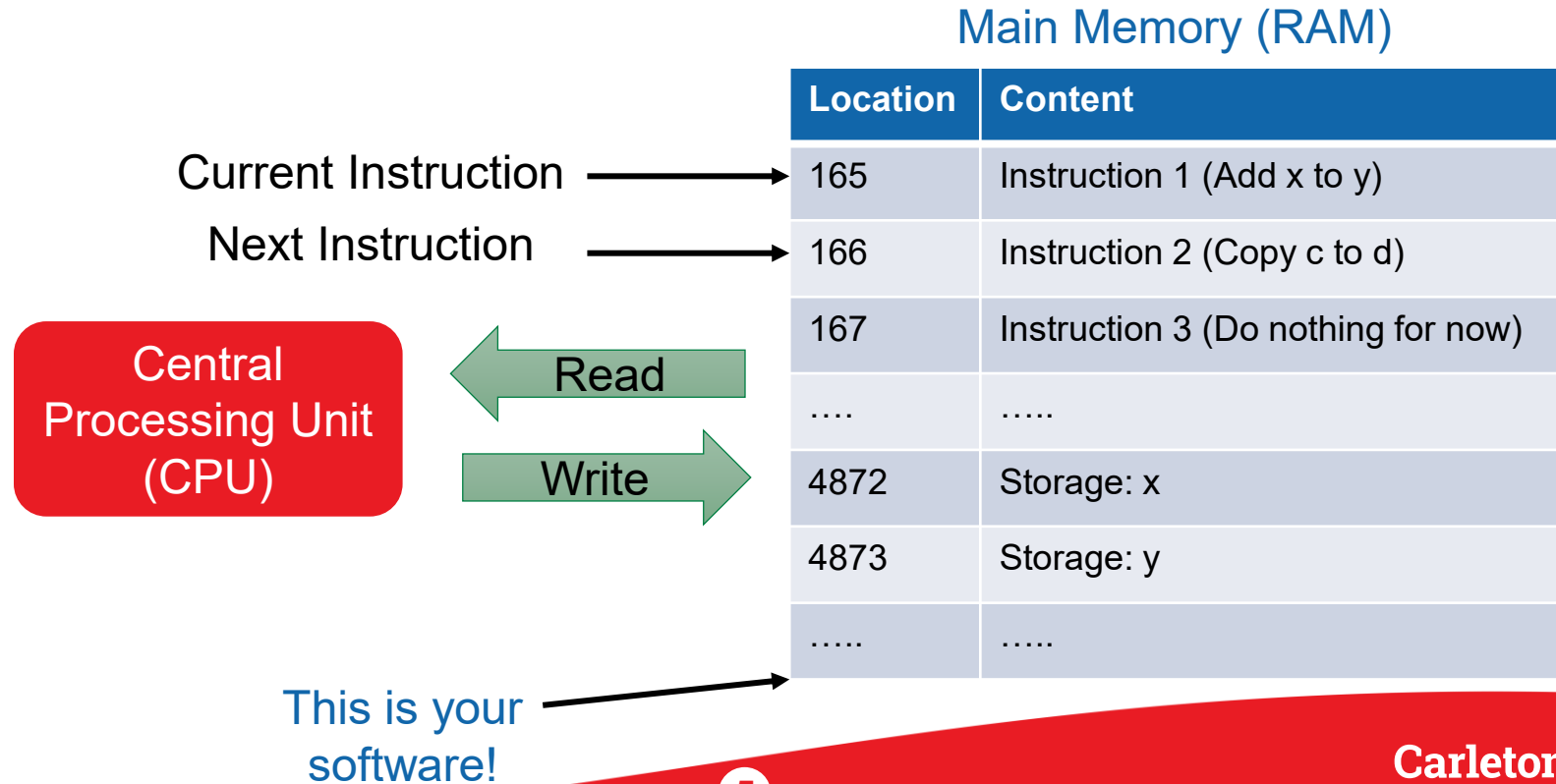


A computer: Hardware & Software

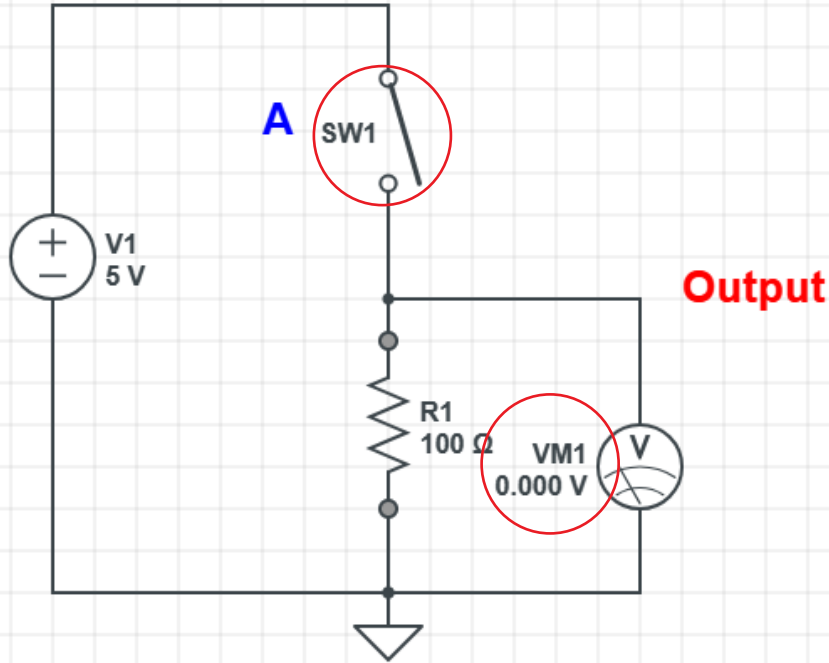
- Hardware (Processor): It is generic, ***programmable***, by software.
- Software: It tells the hardware what to do!



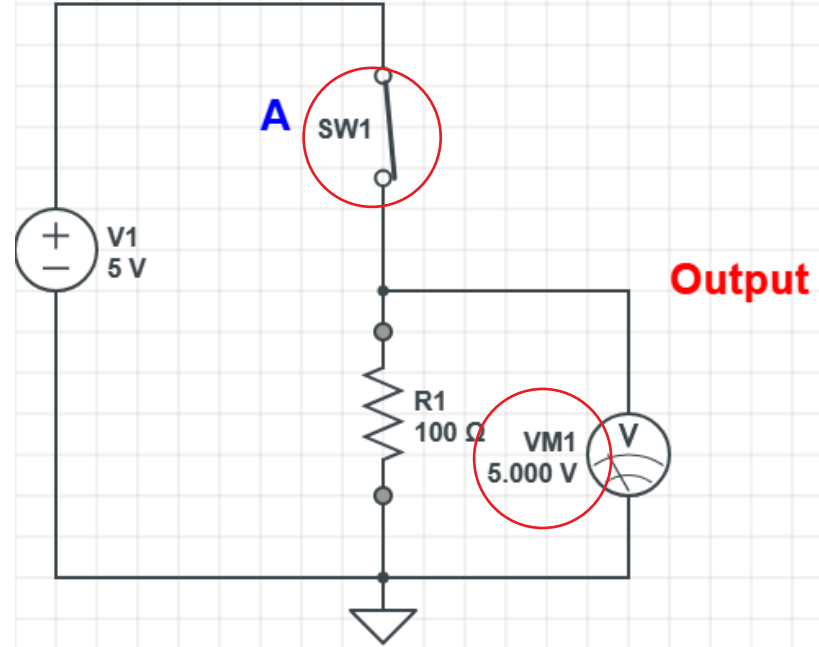
A computer: Hardware & Software



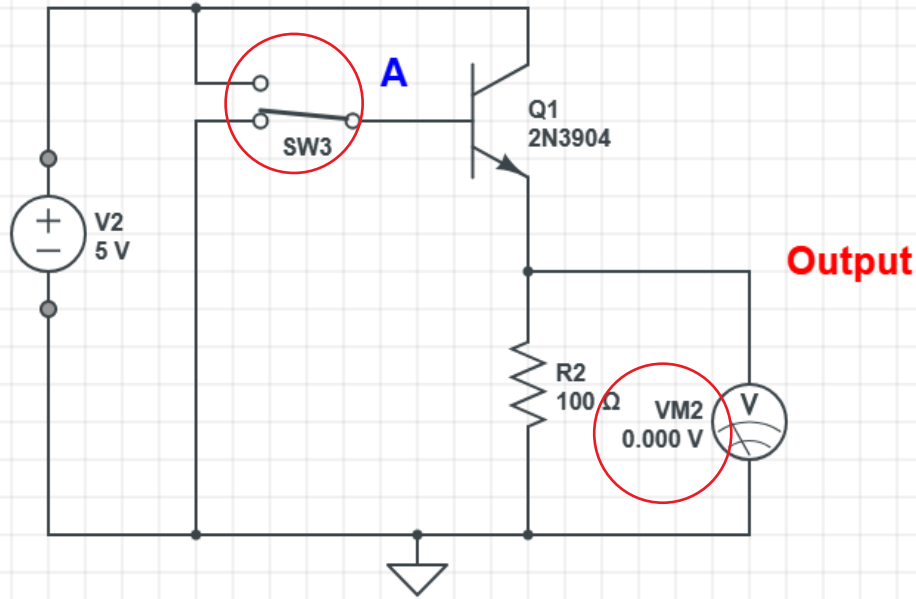
But, how HW can be programmed?



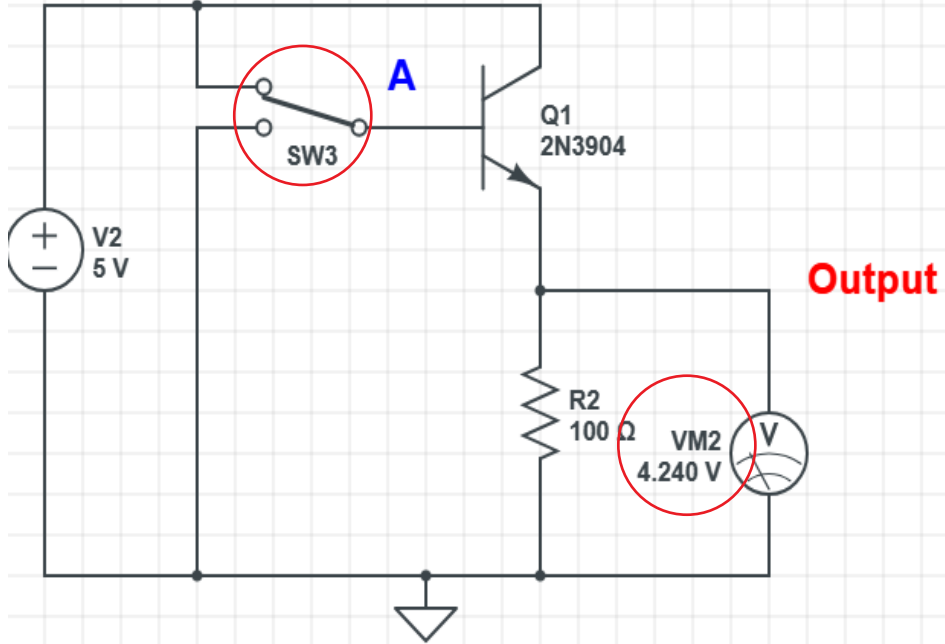
Mechanical Switch



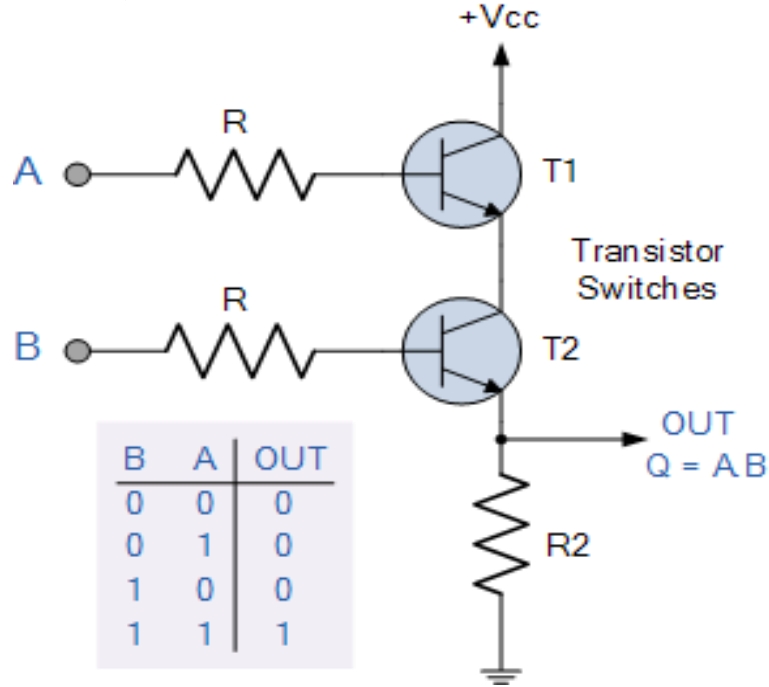
But, how HW can be programmed?



Transistor as a
Switch!

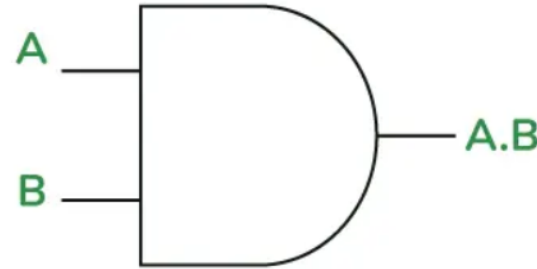


But, how HW can be programmed?



Actual Circuit

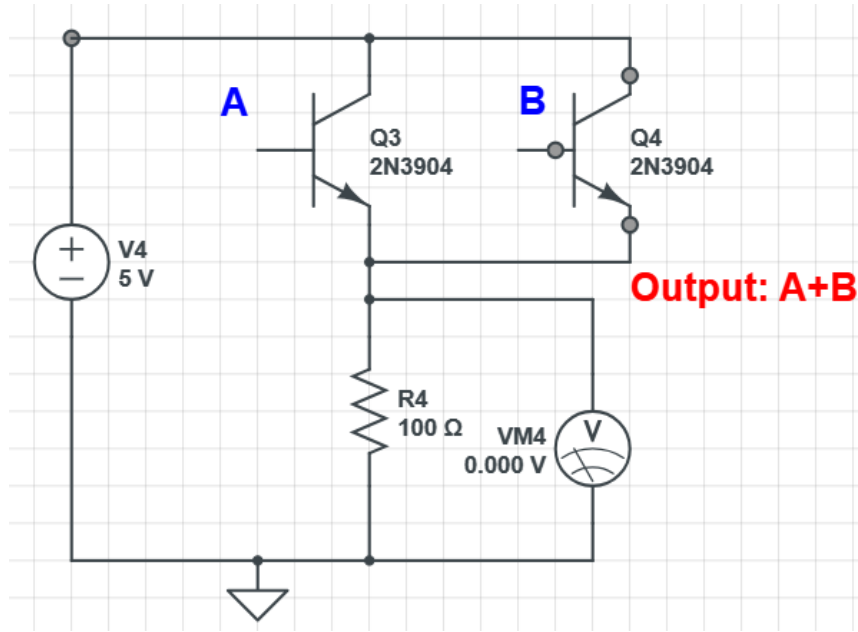
AND Gate



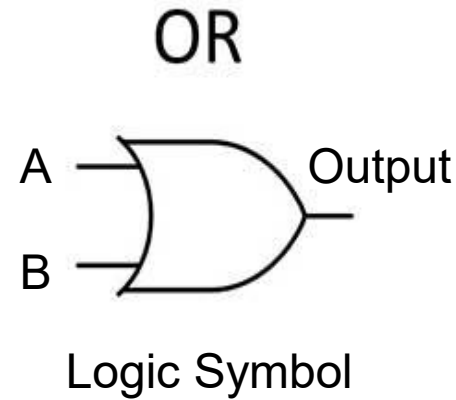
Logic Symbol

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

But, how HW can be programmed?



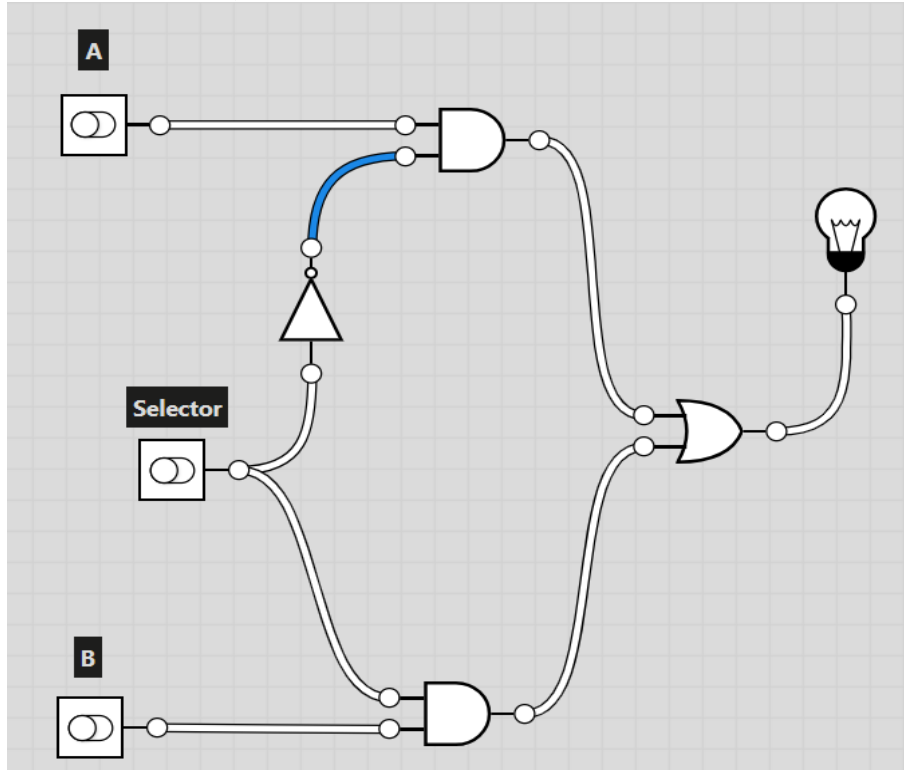
Actual Circuit



A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



But, how HW can be programmed?



Selector/Copier Circuit!

[Logic.ly Online Demo](https://logic.ly/online-demo)

Now, 0s and 1s?

- So the hardware can be manipulated by 0s and 1s
- But, how can we represent our data in 0 and 1?

Simpler than you think

How do you read this number?

275

Exactly!

Two Hundred

Seventy

Five

200

70

5

Simpler than you think

How do you read this number?

In some more detail

275

$$2 \times 100$$

$$2 \times 10^2$$

$$7 \times 10$$

$$7 \times 10^1$$

$$5 \times 1$$

$$5 \times 10^0$$

Why is it 10^x (**powers of 10**)?

How many “**symbols**” do we have?

Big note: **Changing** the “**base**” and number of “**symbols**” **makes different number systems**

Decimal Number Systems

- As we have seen, an integer “*decimal*” number can be thought of as

$$\dots + (d_2 \times 10^2) + (d_1 \times 10^1) + (d_0 \times 10^0) + (d_{-1} \times 10^{-1}) + \dots$$

- Or in more compact form:

$$\sum_{i=-\infty}^{\infty} d_i \times 10^i$$

- We call “**10**” as the **base** of the **decimal** system
 - It defines the “*weights*” of each “*location*”: (ones, tens, hundreds,...)
 - It defines how many “*symbols*” do we have: **0,1,2,3,4,5,6,7,8,9** (ten)

Generic Number Systems

- It does not have to be 10! In general, it can be anything “ r ”

$$\dots + (r_2 \times r^2) + (r_1 \times r^1) + (r_0 \times r^0) + (r_{-1} \times r^{-1}) + \dots$$

- Or in more compact form:

$$\sum_{i=-\infty}^{\infty} r_i \times r^i$$

- We call “ r ” as the **base** of the **generic** system
 - It defines the “**weights**” of each “**location**”: $(r^0, r^1, r^2, \dots)$
 - It defines how many “**symbols**” there are: $0, 1, 2, \dots, r - 1$ (r symbols)

Binary Number Systems

- We can use $r = 2$, which is called the **Binary system**

$$\dots + (b_2 \times 2^2) + (b_1 \times 2^1) + (b_0 \times 2^0) + (b_{-1} \times 2^{-1}) + \dots$$

- Or in more compact form:

$$\sum_{i=-\infty}^{\infty} b_i \times 2^i$$

- We call **“2” as the base** of the **binary** system
 - It defines the **“weights”** of each **“location”**: (1, 2, 4, 8, 16,)
 - It defines how many **“symbols”** do we have: only **0,1!** (two)

Binary Number Systems

- Let's convert from binary to decimal!
- Consider the number:

$$x = (1101)_2$$

- Just as we read 273 as $2 * 10^2 + 7 * 10^1 + 3 * 10^0$, we will read this binary number as:

$$x = (1 * 2^3) + (1 * 2^2) + (0 * 2^1) + (1 * 2^0)$$

$$x = (1 * 8) + (1 * 4) + (0 * 2) + (1 * 1)$$

$$x = 8 + 4 + 0 + 1 = 13$$

Exercises Time!

- Now, convert the following numbers to decimal

Binary	Decimal
$(101)_2$	
$(0110)_2$	
$(1001)_2$	

Letters? in Binary?

- We can encode various digital data types into binary codes
- Not just letters, but images, videos, audio, music, text, ...etc.

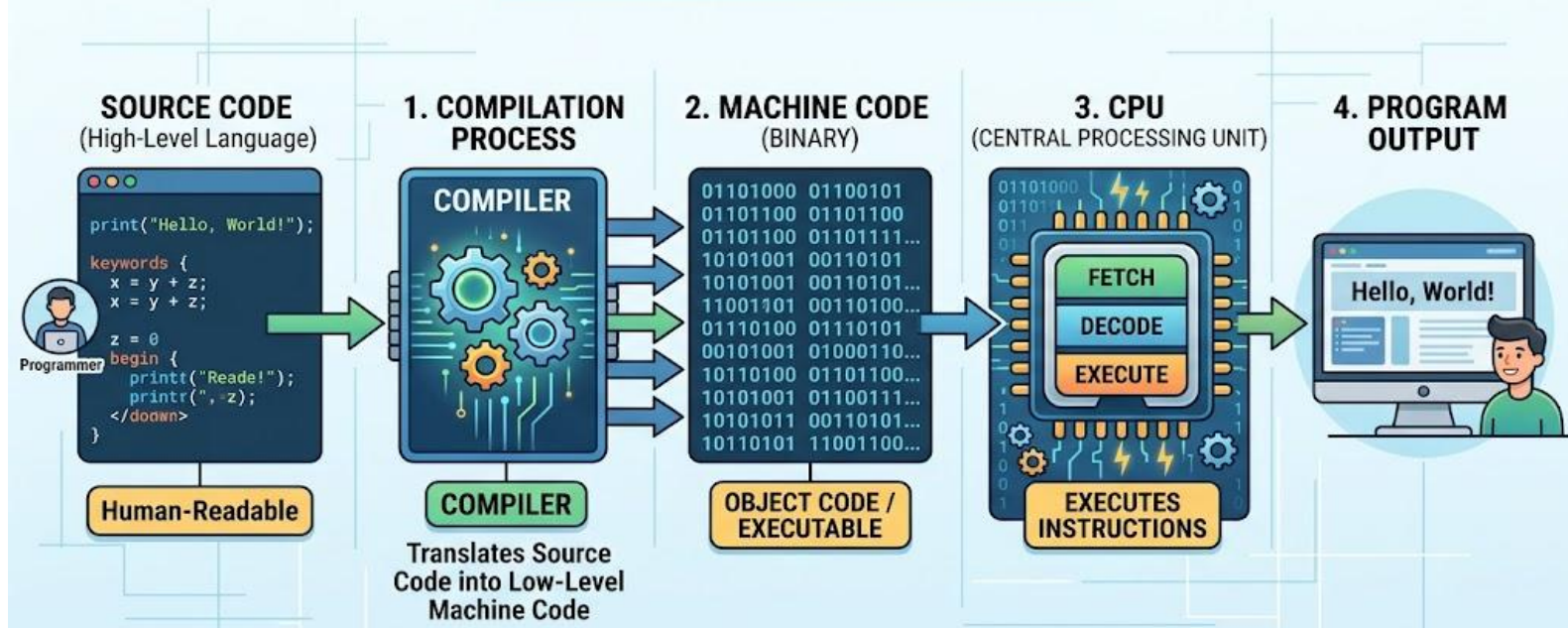
ASCII	Char	Hex	Bin	ASCII	Char	Hex	Bin
65	A	41	0100 0001	97	a	61	0110 0001
66	B	42	0100 0010	98	b	62	0110 0010
67	C	43	0100 0011	99	c	63	0110 0011
68	D	44	0100 0100	100	d	64	0110 0100
69	E	45	0100 0101	101	e	65	0110 0101
70	F	46	0100 0110	102	f	66	0110 0110
71	G	47	0100 0111	103	g	67	0110 0111
72	H	48	0100 1000	104	h	68	0110 1000
73	I	49	0100 1001	105	i	69	0110 1001
74	J	4A	0100 1010	106	j	6A	0110 1010
75	K	4B	0100 1011	107	k	6B	0110 1011
76	L	4C	0100 1100	108	l	6C	0110 1100
77	M	4D	0100 1101	109	m	6D	0110 1101
78	N	4E	0100 1110	110	n	6E	0110 1110
79	O	4F	0100 1111	111	o	6F	0110 1111
80	P	50	0101 0000	112	p	70	0111 0000
81	Q	51	0101 0001	113	q	71	0111 0001
82	R	52	0101 0010	114	r	72	0111 0010
83	S	53	0101 0011	115	s	73	0111 0011
84	T	54	0101 0100	116	t	74	0111 0100
85	U	55	0101 0101	117	u	75	0111 0101
86	V	56	0101 0110	118	v	76	0111 0110
87	W	57	0101 0111	119	w	77	0111 0111
88	X	58	0101 1000	120	x	78	0111 1000
89	Y	59	0101 1001	121	y	79	0111 1001
90	Z	5A	0101 1010	122	z	7A	0111 1010

Now, 0s and 1s?

- We need to write program in 0s and 1s then!
- But this is usually difficult for humans
- So we write in human-like languages, and other “programs” transforms it into 1s and 0s.

From Software to Hardware

TRANSFORMATION OF SOURCE CODE TO MACHINE CODE AND CPU EXECUTION



Break!



So, let's get started

- C/C++: We are targeting the source code phase :)
- Learn about:
 - Syntax
 - Variables
 - Loops
 - Functions

Syntax

- Simple C++ program structure

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    return 0;
}
```

[C++ Syntax](#)

Output

- cout can print numbers and new lines

```
#include <iostream>
using namespace std;

int main() {
    cout << 5 << "\n";
    return 0;
}
```

[C++ Output Challenge](#)

Comments

- Comments do not get executed, by they help the program be more readable.

```
// This is a comment  
cout << "Hello World!";
```

Variables

- Variables are the data containers, and they come in different types
- Let's check [C++ Variables](#)
- Then let's do the challenge [C++ Variables Challenge](#)

Input

- We can use “cin >>” to get input from the user.

```
int x;  
cout << "Type a number: "; // Type a number and press enter  
cin >> x; // Get user input from the keyboard  
cout << "Your number is: " << x; // Display the input value
```

Operations

- Arithmetic [C++ Arithmetic Operators](#)
 - Check integer division
- Assignment [C++ Assignment Operators](#)
 - += operators
- Comparison [C++ Comparison Operators](#)
- Do the simple challenge: [C++ Operators Challenge](#)

If, else !

- Conditional Execution!
 - If something happens
 - Do thing A
 - Else
 - Do thing B
- [C++ If ... Else](#), check if, else, else if, and the coding challenge
- [C++ Conditions Challenge](#)

While loop

- Repeated execution
 - *While something is happening*
 - *Keep doing a thing*
- C++ While Loop
- C++ While Loop Challenge

Functions!

- A block of code that runs only when called.
- Functions are important for code reusing: Some code that will be used multiple times in different places.
 - Define once, use any number of times.
- *Function: takes parameters (Optional)*
- *Do some task*
- *Return a value (Optional)*

[C++ Functions](#)

[C++ Function Parameters](#)

Functions!

- Define a function that takes the radius of a circle and returns its area.
 - Give it the name `circle_area`
- In the main function, define two variables `r1=5.5` and `r2= 3.7`
 - Calculate their areas and store them in variables `A1` and `A2`
- Display the results in the following format:
 - “Circle 1. Radius = ____, Area = ____
Circle 2. Radius = ____, Area = ____

Questions?

